Weekly Student Progress Performa

Team Id	CS26084	**Week no / Date**	Week 3 03 August 2026
Project Title	SentinelCore: A Network Detection and Incident Response Platform		
Members Present	Om Rashiya (24cs084), Yash Patel (24cs082)		
Mentor	Akshita Kadam		

WEEK 3 • Module, Data, and Interface Design

Section	Details to be filled by the team
Planned Tasks (from last week)	Translate the architecture into modules, database models, API routes, navigation, and a 12-week implementation plan.
Tasks Completed This Week	Split the platform into authentication, asset discovery, Suricata sensor, event pipeline, search, correlation, incidents, reports, firewall response, PCAP analysis, and threat intelligence. Defined FastAPI + SQLAlchemy/Alembic, PostgreSQL, Redis, React/Vite, Nginx, and Docker Compose as the implementation stack.
Commits Made	Total: 0 commits (technical design week). Prepared module specifications M03–M12 and acceptance criteria for each milestone.
Issues Created / Closed	Created: A single service would mix privileged capture, API traffic, and long-running ingestion. Closed: Separated API, worker, helper, frontend, proxy, PostgreSQL, and Redis responsibilities.
Milestone Status	~18% complete — on track. Data model and module contracts were ready for implementation.
Blockers / Challenges Faced	The event data volume could make offset pagination and one large table slow. The plan adopted monthly partitions and keyset pagination.
Plan for Next Week	Initialize the repository, implement Docker services, health checks, migrations, and the first authentication screen.

Research and Technical Study

Research objective. Translate the architecture into modules, database models, API routes, navigation, and a 12-week implementation plan.

Study completed. Evaluated synchronous versus asynchronous API/database access, migration strategies, partitioned event storage, and React routing approaches. Async FastAPI and asyncpg were selected for concurrent event/search workloads, while Alembic provides reproducible schema evolution.

Implementation-stack selection

Layer	Selected component	Role in SentinelCore
Web interface	React + Vite	Protected routes and workspaces
Application API	FastAPI + SQLAlchemy	Async API and data access
Durable storage	PostgreSQL + Alembic	Models, partitions, migrations
Coordination	Redis	Offsets and worker coordination
Deployment edge	Docker Compose + Nginx	Reproducible services and routing

Decision carried into the project

Split the platform into authentication, asset discovery, Suricata sensor, event pipeline, search, correlation, incidents, reports, firewall response, PCAP analysis, and threat intelligence. Defined FastAPI + SQLAlchemy/Alembic, PostgreSQL, Redis, React/Vite, Nginx, and Docker Compose as the implementation stack.

Implementation Record and Website Evidence

Module breakdown

Converted the workflow into M1 authentication, M2 application shell, M3 assets, M4 sensor, M5 event pipeline, M6 search, M7 correlation, M8 incidents, M9 reporting, M10 firewall, M11 PCAP, and M12 threat intelligence.

Service responsibilities

Assigned interactive requests to FastAPI, long-running ingestion to a worker, privileged operations to the helper, durable records to PostgreSQL, coordination/checkpoints to Redis, and user interaction to React behind Nginx.

Data strategy

Planned Alembic-managed schemas, monthly event partitions, durable event identifiers, and keyset pagination so the event explorer could operate over growing time-series data.

Access entry point

The sign-in screen became the first designed protected entry point. Its wording communicates that credentials are provisioned by the SentinelCore administrator and that access leads to a secure operations workspace.

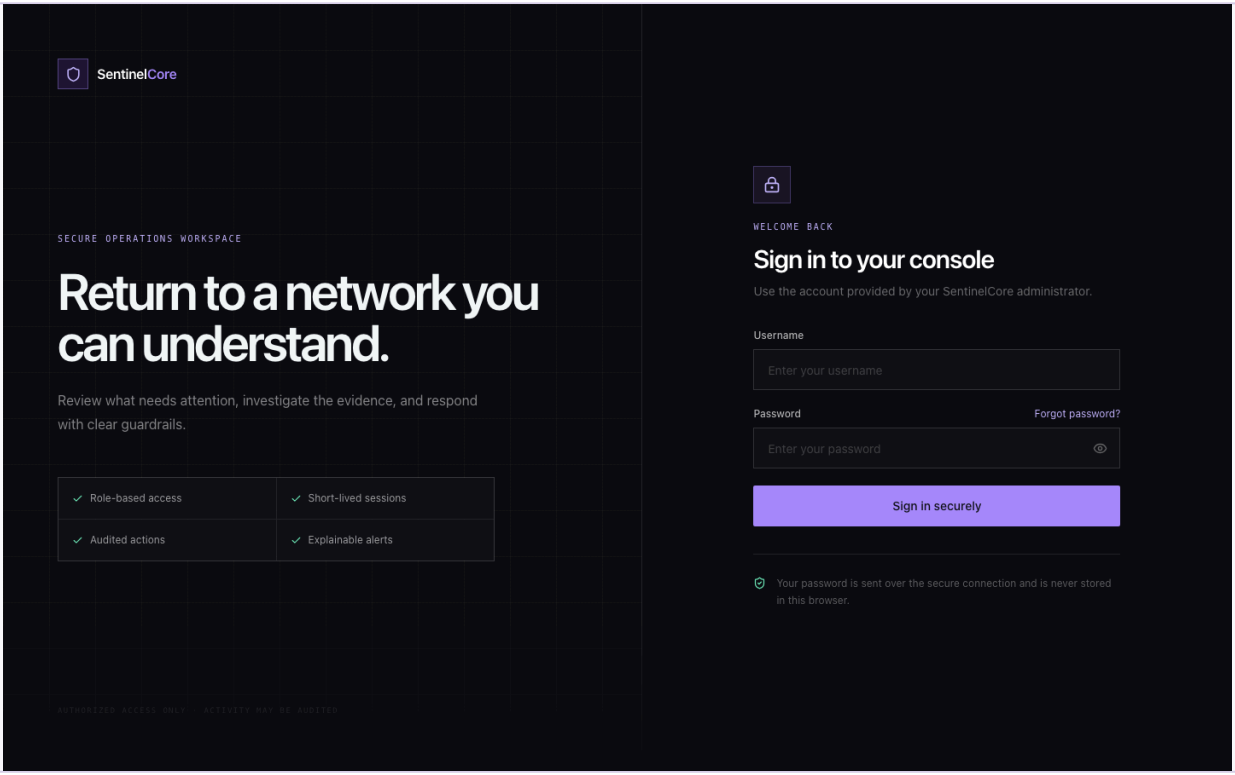


Figure W3.1 — Project-owned SentinelCore interface mapped specifically to Week 3.

Evidence interpretation. The evidence screen shows the implemented sign-in interface, including username/password fields, administrator-provisioned access guidance, and the SentinelCore visual identity. Its visible controls, labels, status fields, and navigation provide direct evidence for the work described on this page.

Repository and Architecture Evidence

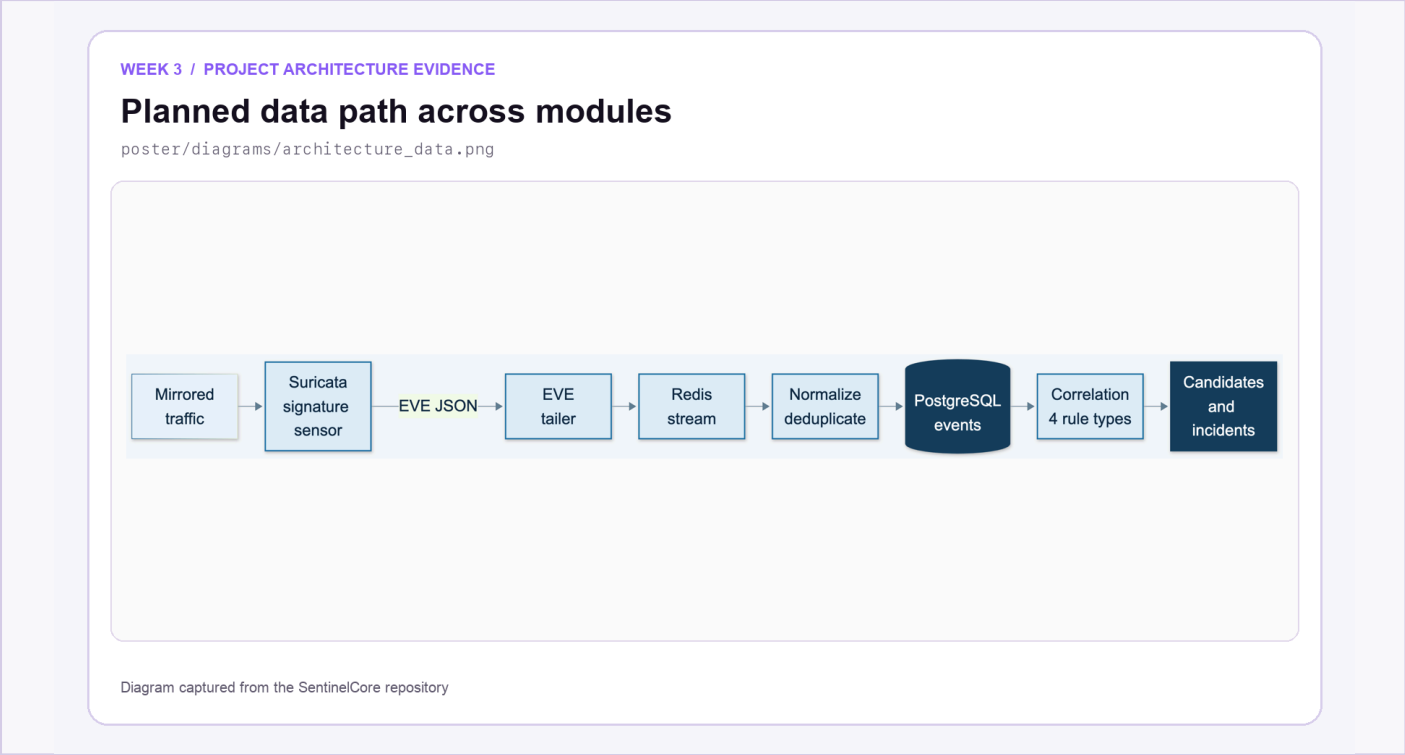


Figure W3.2 — Planned data path across modules. This is a separate internal artifact, not a copy of the website image.

Repository files used for traceability

- reports/modules/M03-asset-discovery_1.md through M12
- frontend/src/App.jsx
- frontend/src/pages/LoginPage.jsx

Verification

Module names, service responsibilities, and navigation were checked against the actual frontend routes, backend models, and repository module specifications.

Traceability conclusion. Planned data path across modules directly supports the Module, Data, and Interface Design milestone. Module names, service responsibilities, and navigation were checked against the actual frontend routes, backend models, and repository module specifications. The artifact, source paths, implementation record, and website evidence therefore describe the same weekly scope.

Weekly Closure and Faculty Assessment

Completed outcome

Split the platform into authentication, asset discovery, Suricata sensor, event pipeline, search, correlation, incidents, reports, firewall response, PCAP analysis, and threat intelligence. Defined FastAPI + SQLAlchemy/Alembic, PostgreSQL, Redis, React/Vite, Nginx, and Docker Compose as the implementation stack.

Commit record

Total: 0 commits (technical design week). Prepared module specifications M03–M12 and acceptance criteria for each milestone.

Issue and resolution

Created: A single service would mix privileged capture, API traffic, and long-running ingestion. Closed: Separated API, worker, helper, frontend, proxy, PostgreSQL, and Redis responsibilities.

Milestone position

~18% complete — on track. Data model and module contracts were ready for implementation.

Challenge handled

The event data volume could make offset pagination and one large table slow. The plan adopted monthly partitions and keyset pagination.

Next-week hand-off

Initialize the repository, implement Docker services, health checks, migrations, and the first authentication screen.

Repository reference

https://github.com/rashiyaom/SentinelCore

Assessed By (Faculty)	Marks Awarded (5 marks)	Faculty Remarks